Comparative Study on the Scalability of Multi-Agent Reinforcement Learning for Workload Offloading in Solar-Powered Edge Networks

DIPARTIMENTO DI INGEGNERIA INFORMATICA AUTOMATICA E GESTIONALE ANTONIO RUBERTI

SAPIENZA
UNIVERSITÀ DI ROMA

Lorenzo Pecorari
1885161

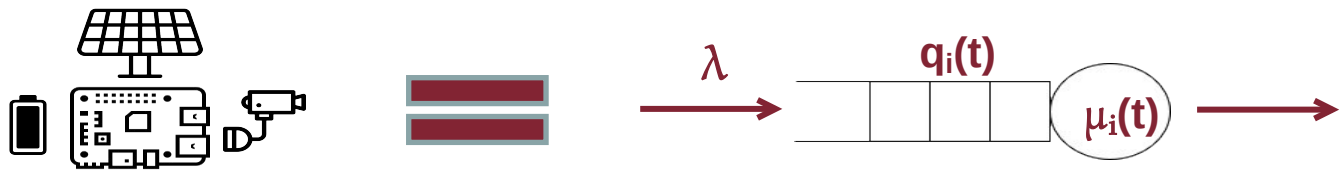
Advisor
Prof. Roberto Beraldi

Introduction

- Solar-powered **edge nodes** with constant workload
- Energy source coming from **solar irradiance**
- Devices usually adopt static thresholds for balancing computational efficiency and energy availability
- Variations on solar energy can compromise operability
- **Reinforcement Learning** as approach to make nodes capable to automatically adapt to different situations

Problem

- **Constant** arrival rate for incoming frames: λ = 15fps
- Maximum computational rate: μ_{max} = 20fps
- **FIFO** queue for storing frames, needed to adapt μ :



- Real data taken from Solcast, referred to year 2024
- Battery capacity **B** in Wh, solar **panel** with A_{PV} in m² and η_{PV}
- **Energy**, in J, from solar light:

$$E_{PV}(t) = \int G(t) A_{PV} \eta_{PV} \, dt$$

- Solar **irradiance** $G(t)$ in W/m²

Energy constraints

- Raspberry Pi 4-like devices with idle power consumption of **2.6W** and **6.0W** at full load
- If local computation only → **3.4W** for frames elaboration
- If local and communications → **3.4W** in 80:20 ratio

- Dynamics of battery energy:

$$E_{Battery}(\mathrm{t}+1) = E_{Battery}(\mathrm{t}) + E_{PV}(\mathrm{t}) - (E_{Frames}(\mathrm{t}) + E_{TXRX}(\mathrm{t}) + E_{Idle})$$

Where $E_{Frames}(\mathrm{t})$, $E_{TXRX}(\mathrm{t})$, E_{Idle} are energy required in t for **processing** frames, **communicating** and **idling** respectively

RL and setup

- Paradigms for describing the problem:
  - **Single-Agent** RL based on MDP
  - **Multi-Agent RL** based on Stochastic games
  - both with independent learners
- Learning algorithms:
  - **Tabular Q-Learning** using multidimensional arrays
  - **Deep Q-Networks** adopting MLP approximators

- Simulation of network from **1** to **10** nodes
- **Python** with Gymnasium, PettingZoo and Stable-Baselines 3
- Agents specifications:
  - **Battery capacities**:

    [25, 100, 50, 37, 65, 80, 40, 75, 55, 90] Wh
  - **PV areas**:

    [1.00, 0.50, 0.75, 0.85, 0.65, 0.55, 0.90, 0.30, 0.80, 0.52] m^2
- Irradiance data of December 21th 2024

Single-agent RL: formulation

- Isolated agents without cooperation
- **Observations**: $o_t^i = [b_t^i, q_t^i, \tau_t^i]$
  - b_t^i as residual battery energy over capacity
  - q_t^i as backlog level on a scale 0-4
  - τ_t^i as timestep over total steps for episode
- **Actions**: $a_t^i \in [0, \mu_{max,i}]$ framerate adopted in t
- Tabular Q-Learning with table size: $|O^i \times A^i| = |O^i| \times |A^i|$
- State discretization for accessing table

Multi-agent RL: formulations

- Agents can use offloading for cooperation
- Different number of agents in the system
- **Action space**: $a_t^i = [f_t^i, x_t^i, g_t^i, h_t^i]$
  - f_t^i as local framerate
  - x_t^i as offloading mode (no off., send, receive)
  - g_t^i as id offloading target
  - h_t^i as offloading framerate
- Different formulation for **observation space**:
  - **Full-state**: all the local state of others
  - **Aggregated-states**: local state min/avg/max of batteries & backlog
  - **Reduced-states**: local state and avg of metrics
  - **Local-only**: local state

Reward function

- Combination of local and offloading actions:

$$R_i(\mathrm{t}) = R_{i\,,\,local}(\mathrm{t}) + R_{i\,,\,offloading}(\mathrm{t})$$

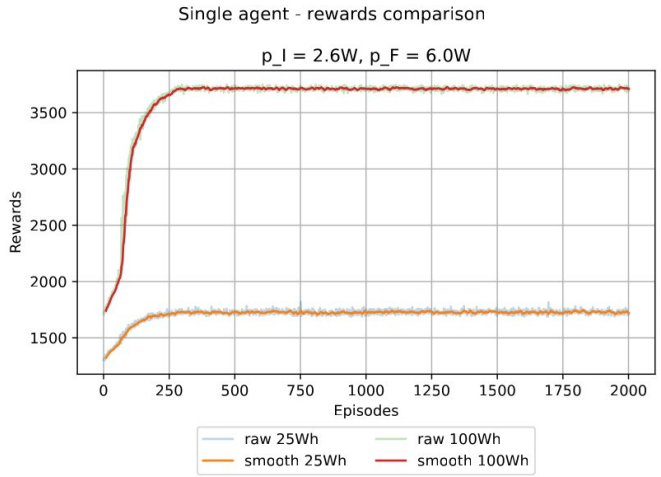
- Each component takes in consideration various metrics as **backlog**, remaining **battery**, computable and comptued **frames**:

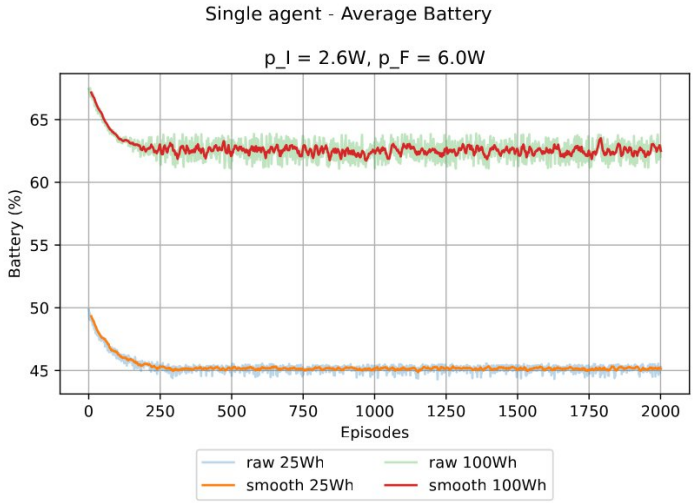
$$R_{i\,,\,*}(\mathrm{t}) = \frac{processed\ frames}{processable\ frames} + \frac{remaining\ battery}{battery\ capacity} + \frac{processed\ frames}{frmaes\ in\ backlog}$$

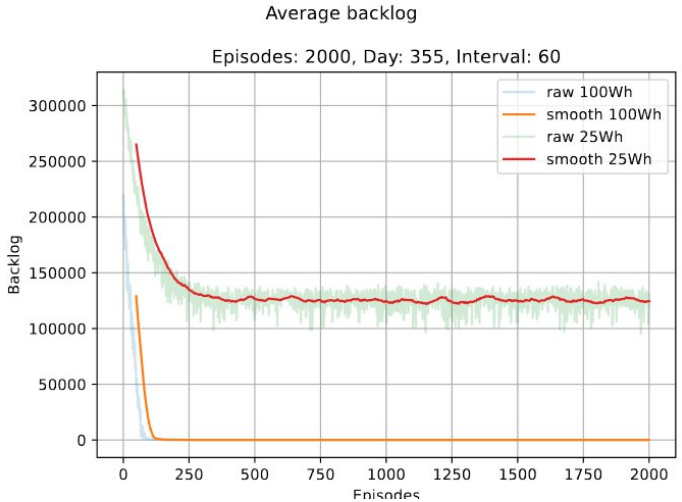
Where:

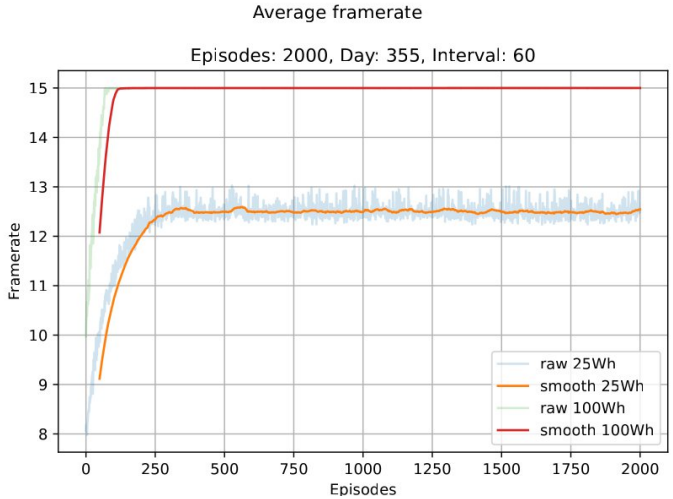
- processable frames = max min (backlog, $\frac{remaining\ battery - E_{idle}}{E_F}$, framerate$\cdot\Delta t$),0)
- processed frames = min(framerate •Δt, processable)

Single-agent RL: experiments

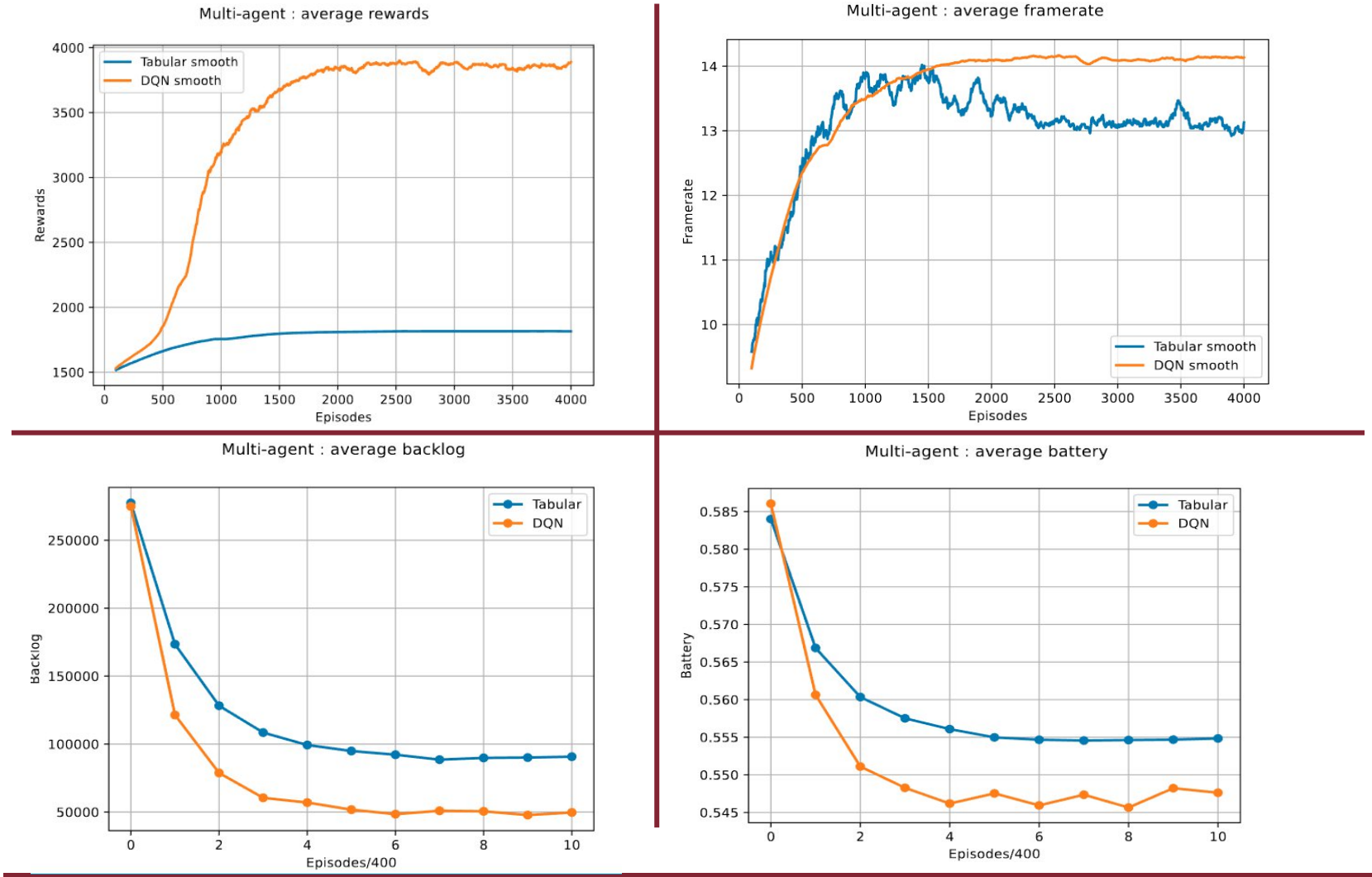




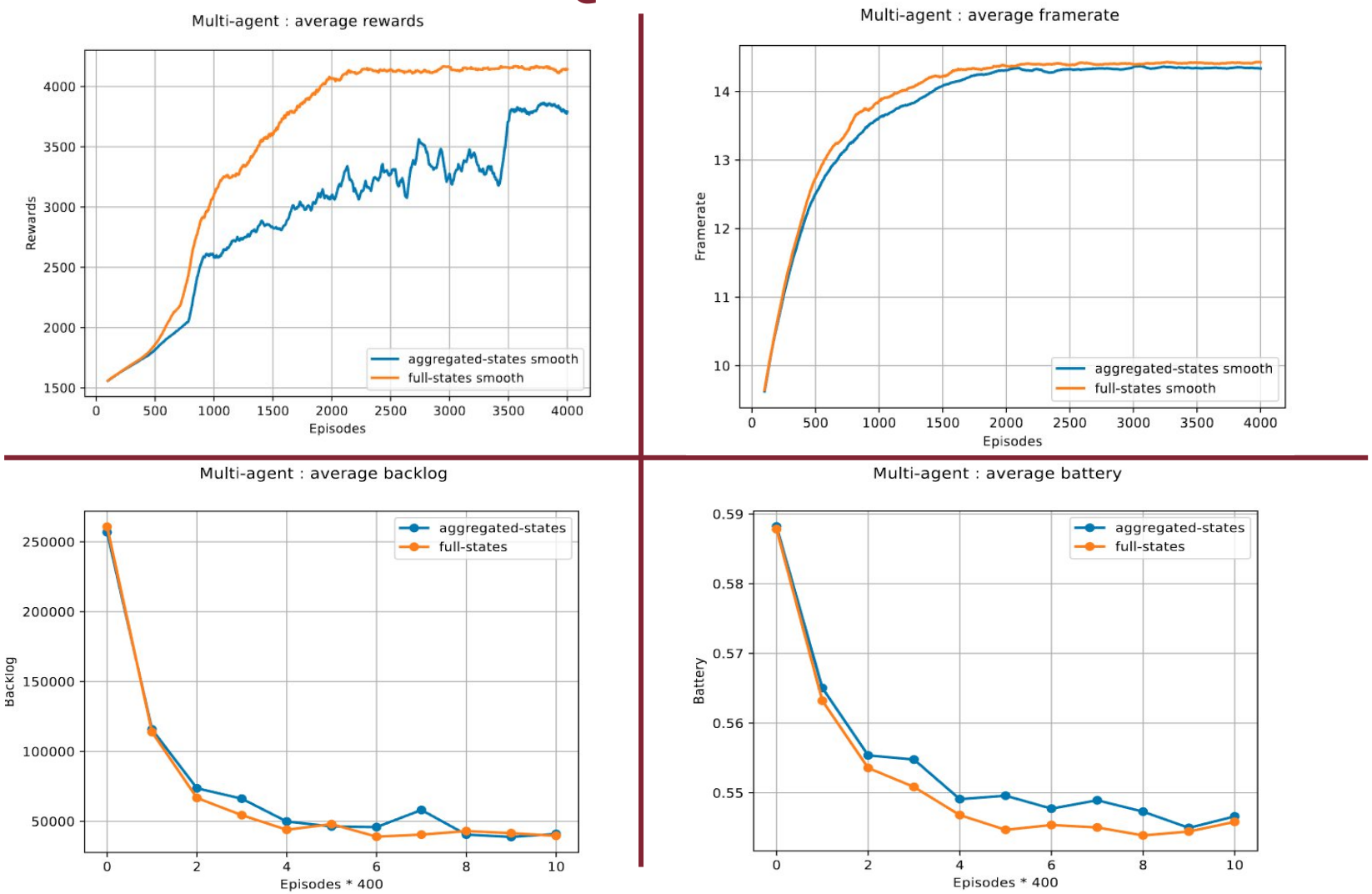




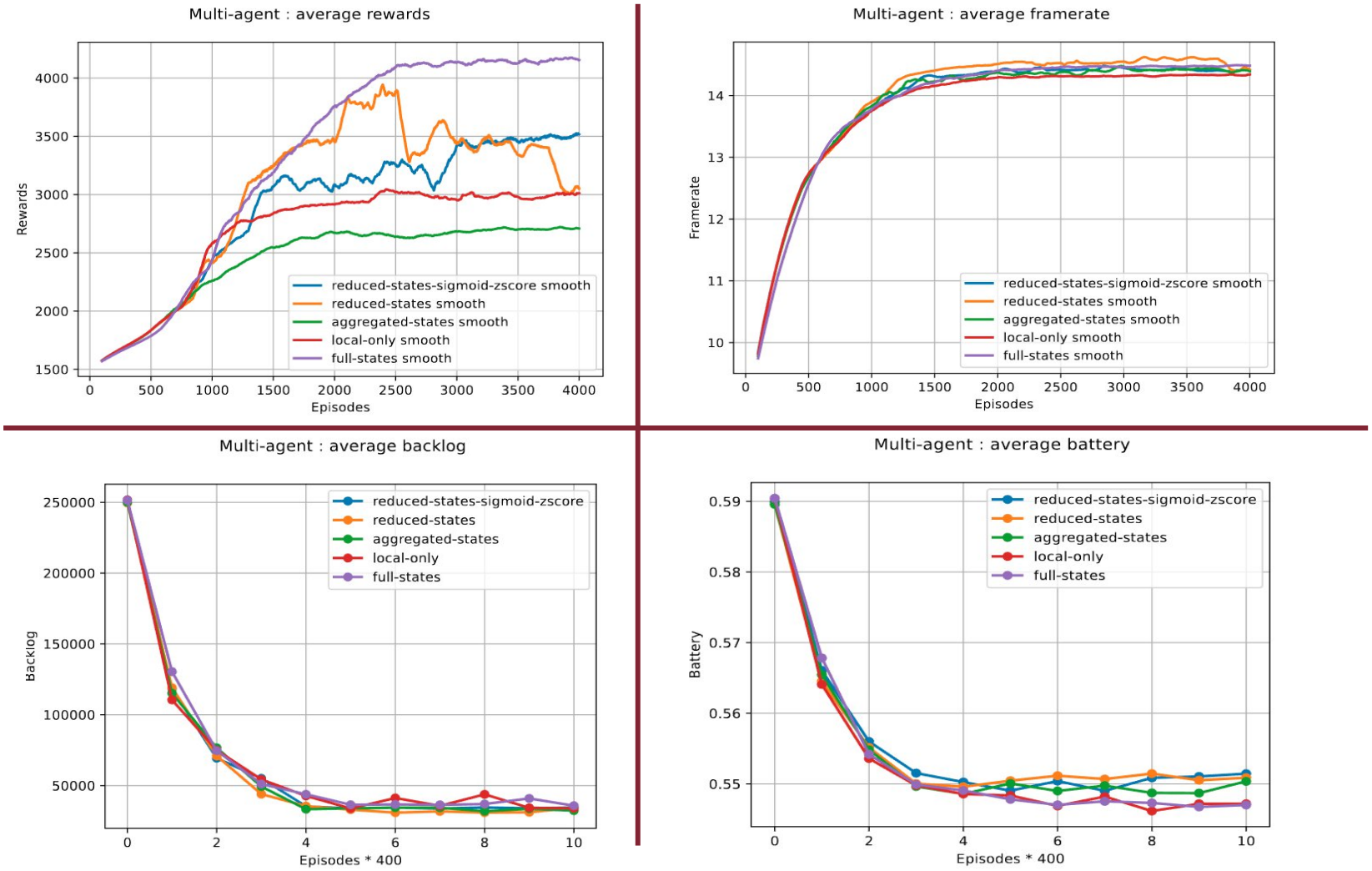
MARL: Tabular-DQN full-state, N=2



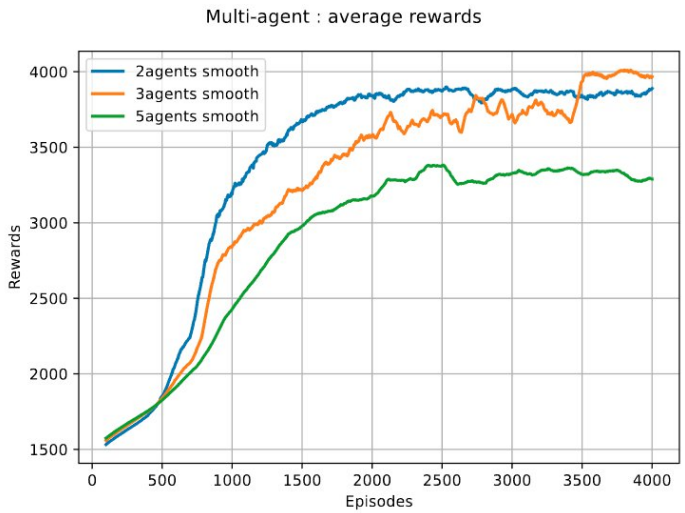
MARL: N=3 - DQN

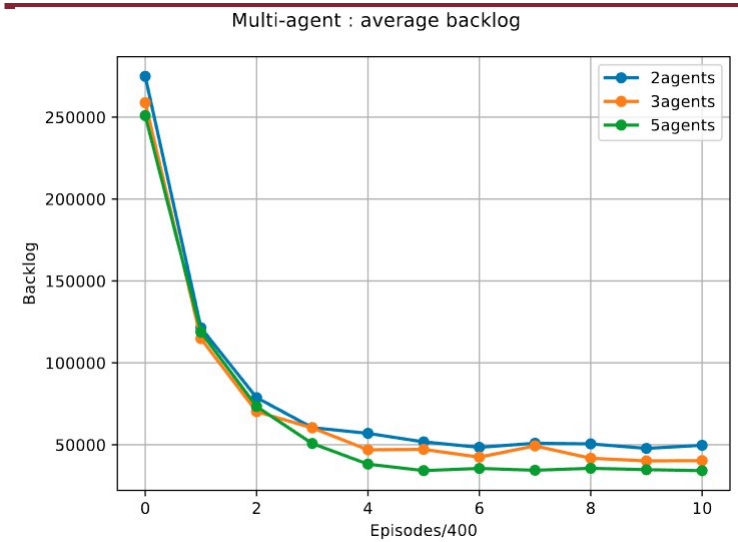

Multi-agent : average rewards
Rewards
Episodes
aggregated-states smooth
full-states smooth
Multi-agent : average framerate
Framerate
Episodes
aggregated-states smooth
full-states smooth
Multi-agent : average backlog
Backlog
Episodes * 400
aggregated-states
full-states
Multi-agent : average battery
Battery
Episodes * 400
aggregated-states
full-states

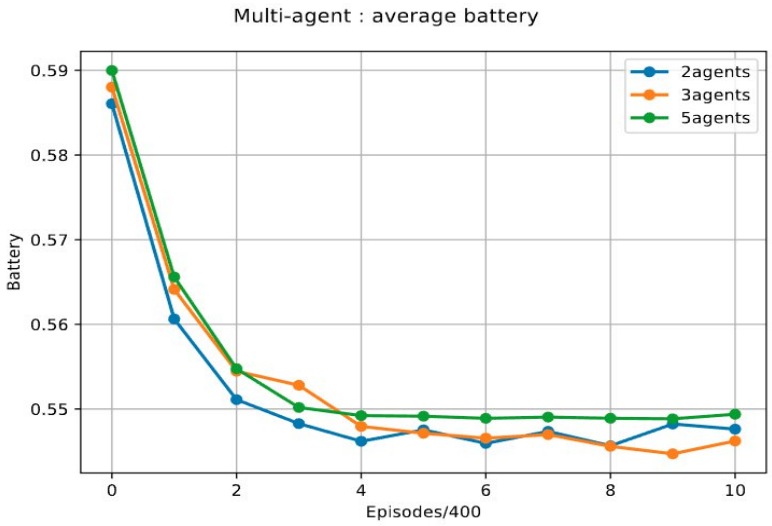
MARL: N=5 - DQN


Multi-agent : average rewards
Rewards
Episodes
reduced-states-sigmoid-zscore smooth
reduced-states smooth
aggregated-states smooth
local-only smooth
full-states smooth
Multi-agent : average framerate
Framerate
Episodes
reduced-states-sigmoid-zscore smooth
reduced-states smooth
aggregated-states smooth
local-only smooth
full-states smooth
Multi-agent : average backlog
Backlog
Episodes * 400
reduced-states-sigmoid-zscore
reduced-states
aggregated-states
local-only
full-states
Multi-agent : average battery
Battery
Episodes * 400
reduced-states-sigmoid-zscore
reduced-states
aggregated-states
local-only
full-states

MARL: 2, 3, 5 agents comparison



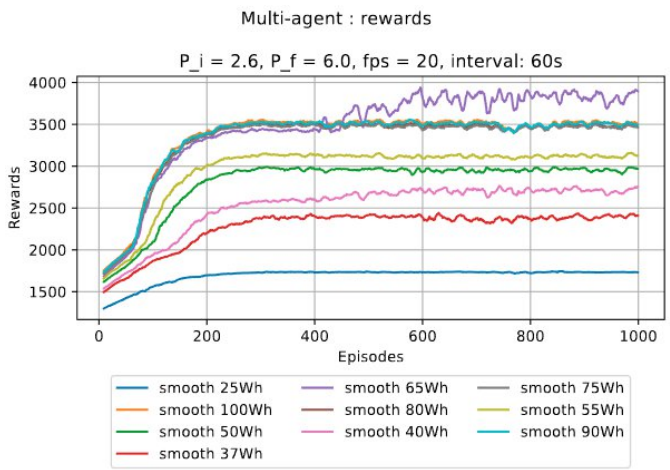


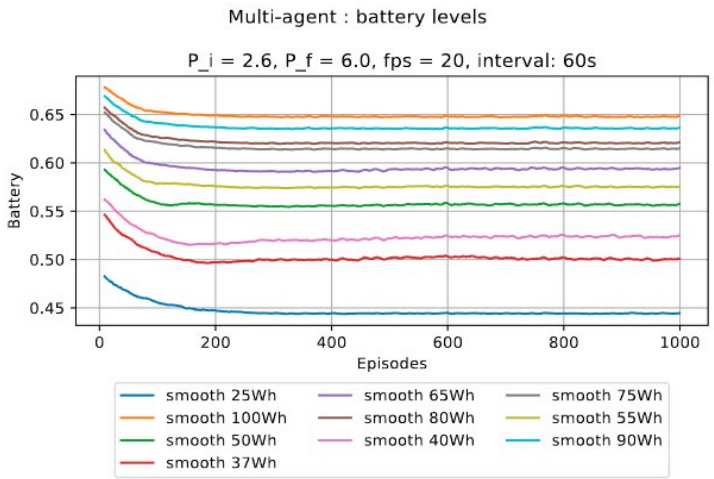


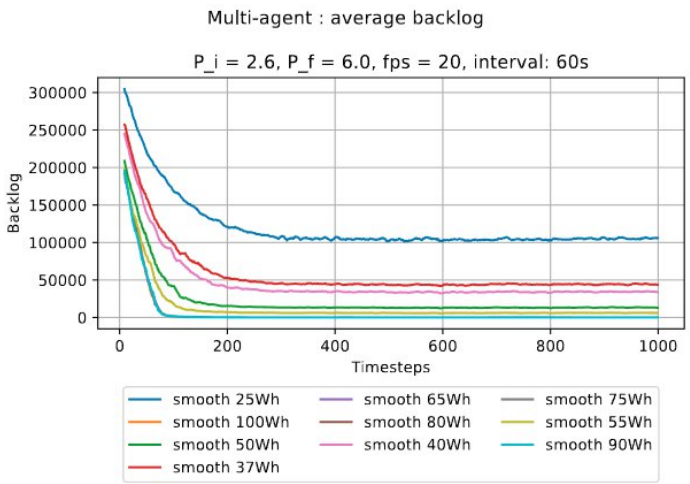
MARL: N=10 - DQN

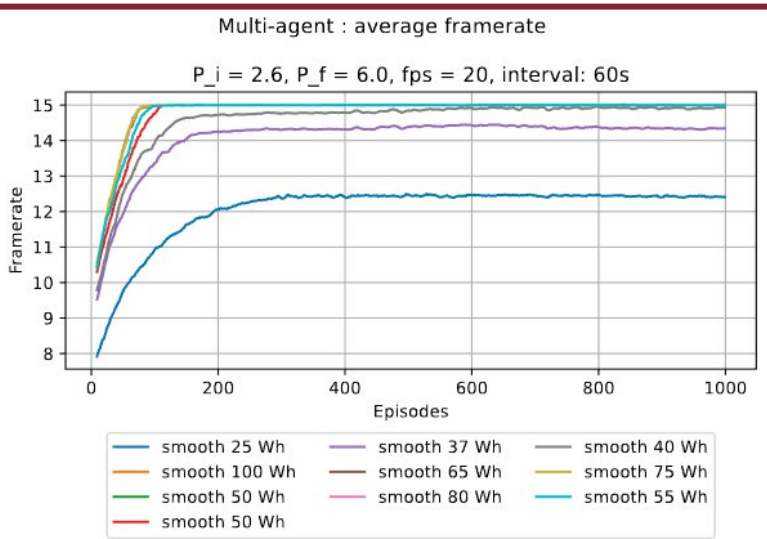
- Experimental approach: training an agent per time with N-1 acting **randomly**
- **Saving** model and retrain another agent **starting** with the pre-trained model
- Repeat until model has not been trained by **all** the agents
- Attempt to obtain a model capable to **generalize** with every node through **fine-tuning**

MARL: N=10 - DQN

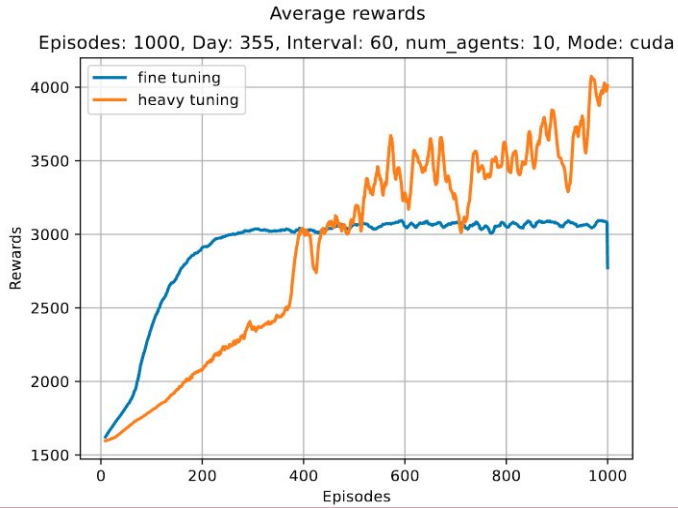

Multi-agent : rewards
P_i = 2.6, P_f = 6.0, fps = 20, interval: 60s
Rewards
Episodes
smooth 25Wh
smooth 100Wh
smooth 50Wh
smooth 37Wh
smooth 65Wh
smooth 80Wh
smooth 40Wh
smooth 75Wh
smooth 55Wh
smooth 90Wh

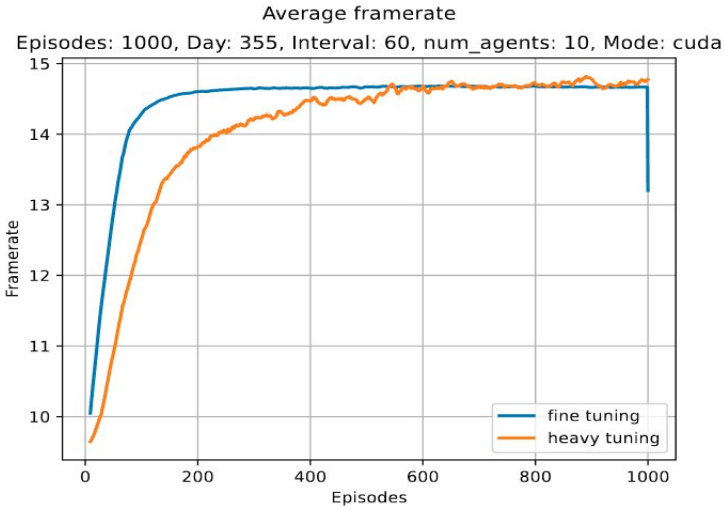

Multi-agent : battery levels
P_i = 2.6, P_f = 6.0, fps = 20, interval: 60s
Battery
Episodes
smooth 25Wh
smooth 100Wh
smooth 50Wh
smooth 37Wh
smooth 65Wh
smooth 80Wh
smooth 40Wh
smooth 75Wh
smooth 55Wh
smooth 90Wh

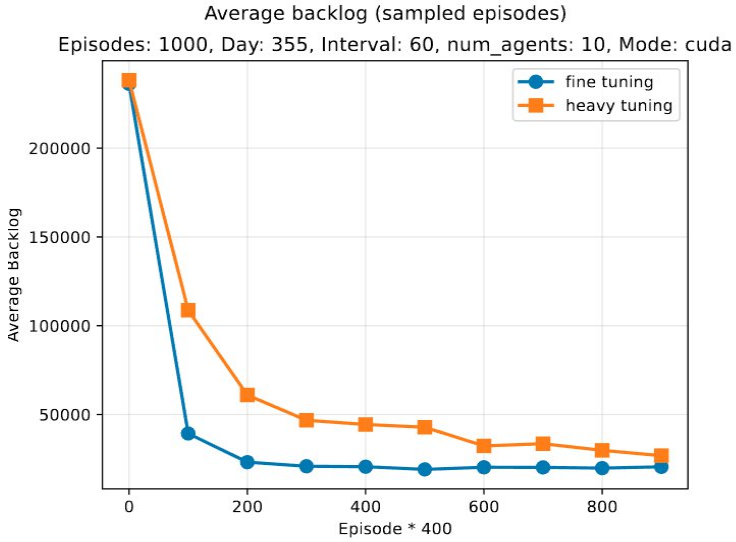

Multi-agent : average backlog
P_i = 2.6, P_f = 6.0, fps = 20, interval: 60s
Backlog
Timesteps
smooth 25Wh
smooth 100Wh
smooth 50Wh
smooth 37Wh
smooth 65Wh
smooth 80Wh
smooth 40Wh
smooth 75Wh
smooth 55Wh
smooth 90Wh

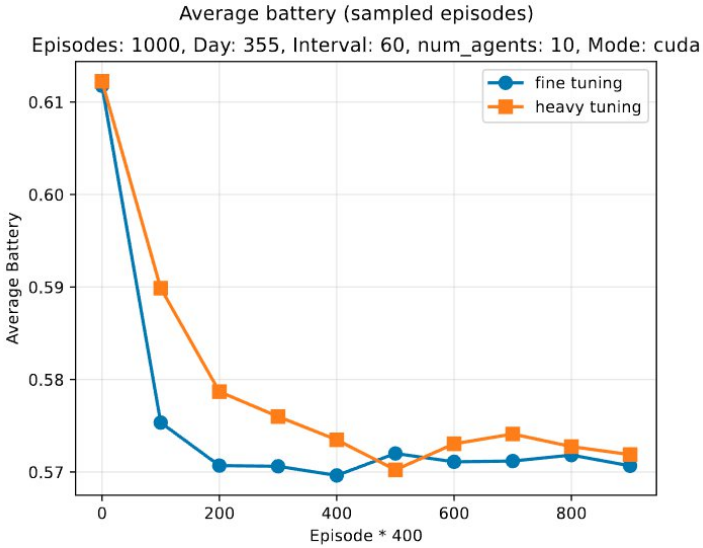

Multi-agent : average framerate
P_i = 2.6, P_f = 6.0, fps = 20, interval: 60s
Framerate
Episodes
smooth 25 Wh
smooth 100 Wh
smooth 50 Wh
smooth 50 Wh
smooth 37 Wh
smooth 65 Wh
smooth 80 Wh
smooth 40 Wh
smooth 75 Wh
smooth 55 Wh

MARL: N=10 - fine vs “heavy” tuning

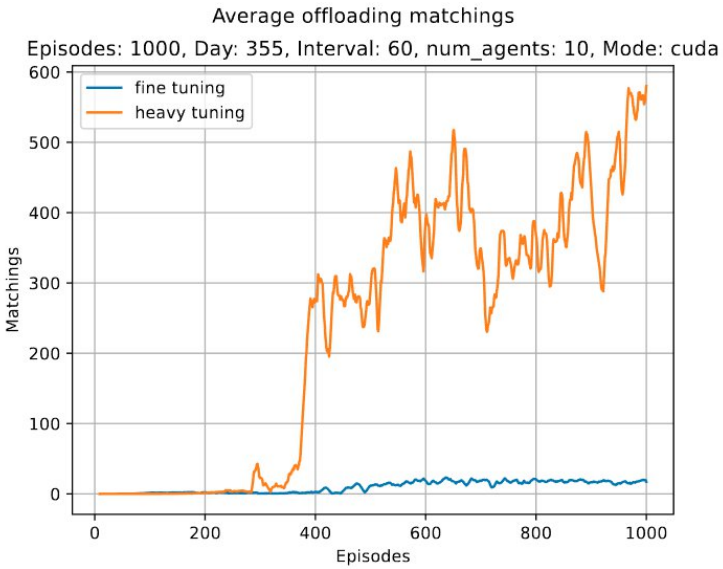








MARL: N=10 - fine vs “heavy” tuning

- Fine tuning remarks higher stability than “heavy” tuning because it privileged **local** computation
- Having N-1 agents acting randomly generates a model that knows that other agents may **not** cooperate

- In average, fine tuning scenario leads to lower offloading matching across the system
- Training from zero has higher reward due to the generation of policies that tend to **cooperate** more